What is a Function?

A **function** is a self-contained, named block of code designed to perform a specific, reusable task. Instead of writing the same logic repeatedly, you wrap it inside a function and invoke it whenever needed.

Key Traits of Functions

Identification: Every function must have a unique identifier name followed by parentheses ().

Execution Rule: A function remains unactive and will **only execute when it is explicitly called**.

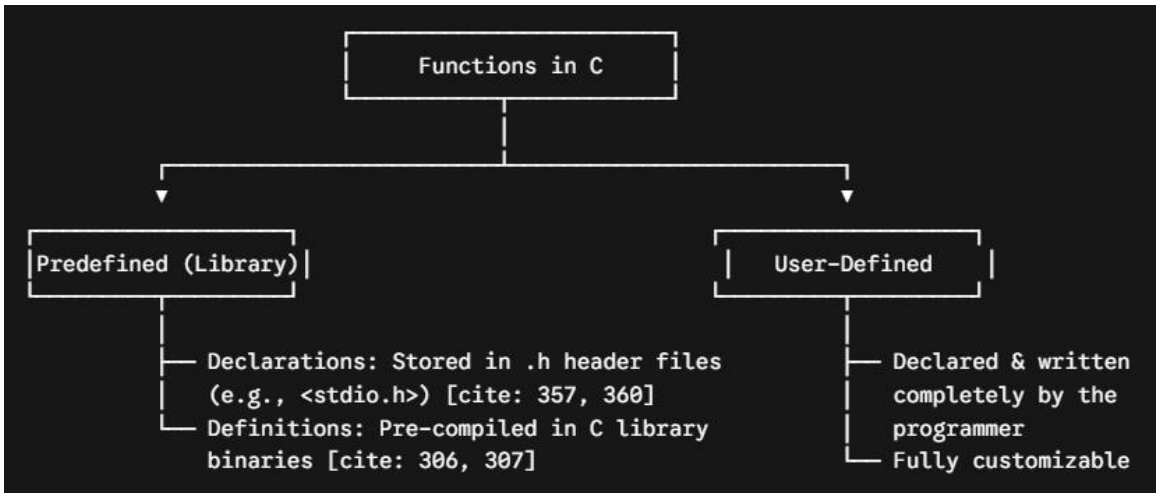
Memory Lifecycle: Whenever a function is called, it is dynamically allocated its own temporary execution frame in the **RAM**.

As soon as the function finishes its work, **its memory is automatically released**.

Execution control jumps straight back to the exact location from which the function was originally invoked.

Classification of Functions

Functions in C are broadly divided into two structural categories:



The Special Role of main()

Every executable C program must contain a main() function.

> You do not manually call main() in your code.
>
> It is invoked directly by the **Operating System (OS)** to kickstart program execution.

The Three Pillars of a Function

To implement a user-defined function properly, you must handle three distinct components:

1. Declaration ----> Tells the compiler a function exit (Ends with ';').

2. Call ----> Executes the function's code block.

3. Definition ----> The actual implementation containing the logic code.

Where do they live? Function declarations is done at the top , while full function definitions are written in source files (.c).

Function Declaration (Prototype)

Tells compiler:

> "A function exists."

Syntax:

> **return_type** function_name (parameters);

Example:

```
int add();
```

Function Definition

Actual implementation of the function.

Example:

```
int add()
{
  return 10+20;
}
```

Function Call

Invoking or executing the function.

Example:

```
add();
```

Complete Example :

```
#include<stdio.h>
int add();                    // Declaration
int main()
{
   int result;
   result = add();            // Function Call
   printf("Sum = %d",result);
   return 0;
}
int add()                     // Definition
{
   return 10+20;
}
```

Output:

Sum = 30

❁ The 4 Approaches to Function Architecture

Depending on how a function handles inputs (**Arguments**) and outputs (**Return Values**), it can be engineered in four distinct ways:

1. Takes Nothing, Returns Nothing (TNRN)

The function operates completely isolated. It takes no data from the caller and sends no data back.

> **Key Indicator:** Uses the **void** keyword as the return type, and has empty parentheses ().
>
> **Rule:** If a function does not return a value, it does not use a return expression keyword.

💻 Code Blueprint

```
#include <stdio.h>

void add();  ----->  Function Declaration (Prototype)

int main()

{

   add();  ----->  Function Call

   return 0;

}

void add() ----->  Function Definition { 'void' means Returns Nothing }

{

   int a, b, sum = 0;  -----> Takes Nothing: values are captured internally

   printf("Enter two numbers: ");

   scanf("%d %d", &a, &b);

   sum = a + b;

   printf("Sum = %d\n", sum); ----->  Displayed directly inside the function

}
```

2. Takes Something, Returns Nothing (TSRN)

The caller passes data into the function, but the function prints or processes the results locally without returning anything back.

💻 Code Blueprint

```
#include <stdio.h>

void add(int, int);  ----->  Declaration: specifies the input data types

int main()

{

   int x, y;

   printf("Enter two numbers: ");

   scanf("%d %d", &x, &y);
```

```
    add(x, y);  ----->  Function Call with "Actual Arguments"
    return 0;
}
void add(int a, int b)  -----> Function Definition with "Formal Arguments"
{
    int c = a + b;  ----->  Uses data copied from x and y
    printf("Sum is %d\n", c);
}
```

🎯 **Important Vocabulary:**

* **Actual Arguments:** The real variables/values passed into a function during its call (x, y).

* **Formal Arguments:** The local placeholder variables assigned in the function definition header to receive those copies (a, b).

3. Takes Nothing, Returns Something (TNRS)

The function collects data independently but delivers the calculated result back to the caller.

💻 Code Blueprint

```
#include <stdio.h>
int add();  ----> Declaration: returns an integer value
int main()
{
    int s;
    s = add();  ----> Function Call: returned value is collected and assigned to 's'
    printf("Sum is %d\n", s);
    return 0;
}
int add()
```

```
{
   int a, b, c;
   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);
   c = a + b;
   return c;  ----> Execution control and the value of 'c' bounce back to 's'
}
```

4. Takes Something, Returns Something (TSRS)

The most modular and dynamic function style. It accepts data values, computes them, and passes the result back out to the calling function.

💻 Code Blueprint

```
#include <stdio.h>
int add(int, int);  ----->  Declaration
int main()
{
   int x, y, s; // [cite: 424]
   printf("Enter two numbers: ");
   scanf("%d %d", &x, &y);
   s = add(x, y);  ----->  Passes (x, y) and receives the computed value back into 's'
   printf("Sum is %d\n", s);
   return 0;
}
int add(int a, int b)
{
   int c = a + b;
   return c ;  ----->  Returns value back
}
```

💡 Practical Rules & Cheat Sheet

The Semicolon Rule: Always end function *declarations* with a semicolon ; (e.g., void add();). **Never** put a semicolon after a function *definition* header block.

The void Rule: If you don't want a function to pass any data back to its caller, prefix its name with the void return type.

The Print Rule: As notes in your text point out, if a function directly displays or outputs its information using a printf() inside its block, it often naturally shifts towards a **Return Nothing (void)** model design.

📖 **Stuck or Prefer a Visual Walkthrough?** If you are not able to understand through these notes or you need to explore these concepts more visually, you can review my handwritten notes PDF provided in the same folder: **08 Functions.**